

You are the principal engineer responsible for this repository.

The application is a multi-tenant Canadian Smith Manoeuvre automation simulator. It simulates a readvanceable mortgage, HELOC, ordinary bank account, brokerage account, transfers, and investment orders.

The system automatically performs the basic monthly debt-conversion process:

1. A Temporal Schedule starts a monthly conversion workflow shortly after the expected mortgage-payment date.
2. The workflow checks the simulated bank for the actual mortgage payment.
3. It waits until the payment is SETTLED.
4. It reads the actual principal repaid.
5. It waits until the HELOC available credit has increased.
6. It draws no more than the principal repaid, newly available HELOC credit, user monthly cap, and platform cap.
7. It transfers the HELOC funds to the non-registered brokerage account.
8. It waits for the brokerage deposit to settle.
9. It submits a fractional market purchase for the user-configured simulated ETF.
10. It confirms the order fill.
11. It reconciles the complete money trail.
12. It creates an immutable audit record.
13. It pauses the strategy when any safety condition or reconciliation rule fails.

The HELOC interest payment is separate:

- HELOC interest is paid from the user's ordinary bank account.
- Investment funds must never be used to pay HELOC interest.
- The system monitors and reconciles the interest debit separately.

Technology:

- Node.js 20 or later
- TypeScript with strict mode
- pnpm workspaces
- Temporal TypeScript SDK
- PostgreSQL
- Prisma
- Fastify
- Next.js
- Docker Compose
- Vitest
- Zod
- OpenAPI
- Structured JSON logging

Architecture rules:

- Keep Temporal Workflow code deterministic.
- Workflows must not perform database access, network access, random-number generation, filesystem access, or direct external API calls.
- All external operations must run in Temporal Activities.
- Use integer cents for CAD money.
- Use Decimal or string values for security quantities.
- Never use floating-point arithmetic for money.
- Every user-owned table must contain tenantId.
- Every repository method must require tenantId.
- Never trust tenantId supplied only through request bodies.
- Derive tenant context from authenticated identity.
- All external financial POST operations require idempotency keys.
- A timed-out financial POST must be reconciled before retrying.
- Do not blindly retry transfers or investment orders.
- Workflows orchestrate; domain services enforce business invariants.
- Simulators must support deterministic scenarios for automated tests.
- Simulators may additionally support seeded pseudo-random failures for demos.
- Use database constraints as a final defence against duplicates.
- Never store access tokens or credentials in Temporal Workflow input or history.
- Do not log account credentials, tokens, full account numbers, or sensitive personal data.
- Store provider payloads only after redaction.
- Produce append-only financial ledger records.
- Do not hide leverage, debt, interest, or investment risk in customer-facing copy.

Implementation process:

1. Inspect the existing repository before changing code.
2. State assumptions.
3. Produce a concise implementation plan.
4. Identify files that will be created or changed.
5. Implement one bounded phase at a time.
6. Run formatting, linting, type checking, unit tests and relevant integration tests.
7. Fix failures before declaring completion.
8. Do not replace functional code with placeholders.
9. Do not leave TODO comments unless clearly listed as intentionally deferred.
10. Never claim a command passed unless it was executed successfully.

Code-quality expectations:

- Explicit interfaces at application boundaries.
- Zod validation for untrusted input.
- Dependency injection for external adapters.
- Small, testable domain services.
- Exhaustive state handling.
- Typed errors with stable machine-readable codes.
- Idempotent consumers and webhook handlers.
- Correlation IDs across API, workflow, activity, bank and brokerage operations.
- UTC storage, explicit IANA timezone handling for schedules.
- Database migrations committed to the repository.
- No destructive migration without explicit warning.

Read CLAUDE.md or the Cursor project rules before doing anything.

We are starting a new multi-tenant Smith Manoeuvre automation simulator.

Do not write application code yet.

Create the initial architecture documentation:

1. docs/product-requirements.md
2. docs/system-architecture.md
3. docs/monthly-conversion-workflow.md
4. docs/heloc-interest-workflow.md
5. docs/security-and-tenancy.md
6. docs/failure-model.md
7. docs/domain-glossary.md
8. docs/decisions/ using Architecture Decision Records

The normal monthly workflow is:

- A Temporal Schedule starts the workflow after the user's expected mortgage-payment date.
- The Schedule does not assume the payment has settled.
- The workflow polls the bank every six hours.
- Later, a bank webhook may signal the workflow to check immediately.
- The workflow proceeds only when the mortgage payment is SETTLED.
- It obtains the actual principal amount from the bank.
- It waits until the HELOC available credit reflects the principal reduction.
- It calculates the investment amount as the minimum of:
  - actual principal repaid,
  - newly available HELOC credit,
  - user monthly cap,
  - platform monthly cap.
- It draws the HELOC amount.
- It confirms the draw before proceeding.
- It transfers the funds to a non-registered brokerage account.
- It confirms the deposit.
- It buys a user-selected simulated ETF using a notional fractional market order.
- It confirms the fill and settlement.
- It reconciles all operations.
- It completes the cycle or automatically pauses the strategy.

HELOC interest:

- The bank charges interest to the HELOC.
- The interest payment is debited from the user's ordinary bank account.
- A separate workflow confirms the charge, debit and settlement.
- The investment conversion must not use HELOC funds to pay interest.

Design the following:

- Bounded contexts
- Service boundaries
- Temporal Workflow and Activity boundaries
- Data ownership
- Account and transaction model
- Multi-tenant security model
- Idempotency model
- Reconciliation model
- State machines
- Failure taxonomy
- Observability strategy
- Audit and money-tracing strategy
- Simulator scenario model
- Testing pyramid
- Local Docker architecture
- Production evolution path

Create Mermaid diagrams for:

- System context
- Container architecture
- Monthly workflow
- Interest-payment workflow
- Transfer sequence
- Reconciliation sequence
- Failure recovery
- Tenant isolation

Create ADRs for:

- Temporal Schedule instead of operating-system cron
- Durable polling plus optional webhook Signal
- PostgreSQL
- Modular monolith with separate deployable processes
- Integer cents for money
- Append-only ledger
- One workflow per monthly conversion cycle
- One long-lived strategy coordinator versus Schedule-started independent cycles
- At-most-one active cycle per strategy and payment period

For the coordinator decision, compare both options and recommend one. Prefer independent monthly cycle workflows started by a Temporal Schedule unless a long-lived entity workflow provides a clear benefit.

Include explicit non-goals:

- Real bank integration
- Real brokerage integration
- Investment advice
- Tax determination
- Portfolio rebalancing
- Dividend recycling
- Capitalizing HELOC interest
- Real-money execution

Finish with:

- proposed monorepo structure,
- implementation order,
- unresolved questions,
- acceptance criteria for the complete MVP.

Read all project documentation and ADRs.

Create the project scaffold, but do not implement the financial workflows yet.

Use this monorepo structure:

```
apps/  
  api/  
  worker/  
  bank-simulator/  
  brokerage-simulator/  
  web/
```

```
packages/  
  contracts/  
  domain/  
  database/  
  observability/  
  temporal-workflows/  
  temporal-activities/  
  bank-client/  
  brokerage-client/  
  test-support/
```

- Use:
- pnpm workspaces
  - TypeScript strict mode
  - Node.js 20+
  - Fastify for API and simulators
  - Next.js for web
  - Prisma with PostgreSQL
  - Vitest
  - ESLint
  - Prettier
  - Zod
  - Pino structured logging

- Create:
- root package scripts
  - shared TypeScript configuration
  - lint and formatting configuration
  - Dockerfiles for every application
  - docker-compose.yml
  - docker-compose.test.yml
  - .env.example
  - health endpoints
  - container health checks
  - startup dependency checks
  - database migration commands
  - Temporal Worker bootstrap
  - Temporal client bootstrap
  - graceful shutdown
  - correlation-ID middleware
  - basic OpenAPI generation

- Docker Compose services:
- postgres
  - temporal
  - temporal-ui
  - api
  - worker
  - bank-simulator
  - brokerage-simulator
  - web

- Requirements:
- Pin all Docker image versions. Do not use latest.
  - Do not hardcode production credentials.
  - Local credentials may come from .env.example.
  - Containers must communicate through Docker service names.
  - Browser-exposed URLs must use localhost.
  - Add health checks and dependency conditions where supported.
  - All services must produce JSON logs.
  - Run database migrations through an explicit command or migration service.
  - Do not have several application containers race to apply migrations.

- Create a README with exact commands for:
- installation,
  - startup,
  - migrations,
  - Temporal UI,
  - running tests,
  - stopping and deleting local volumes.

- Run:
- pnpm install
  - formatting
  - linting
  - TypeScript checking
  - available unit tests
  - docker compose config

Fix all failures before completing.

Read the architecture documentation and existing scaffold.

Design and implement the domain model and Prisma schema.

Entities:

Tenant  
User  
Strategy  
StrategyInvestmentPolicy  
FinancialConnection  
FinancialAccount  
Mortgage  
Heloc  
BrokerageAccount  
OrdinaryBankAccount  
MortgagePayment  
HelocCreditEvent  
HelocInterestCharge  
HelocInterestPayment  
MonthlyConversionCycle  
MoneyMovement  
BrokerageDeposit  
InvestmentOrder  
InvestmentFill  
LedgerEntry  
Reconciliation  
ReconciliationItem  
IdempotencyRecord  
ProviderWebhookEvent  
OperationalException  
AuditDocument  
WorkflowReference

Requirements:

1. Every user-owned entity must contain tenantId.
  2. Use UUID or ULID identifiers consistently.
  3. All CAD monetary values use integer cents.
  4. Security quantity and price precision must be represented safely.
  5. Store timestamps in UTC.
  6. Strategy has a Canadian IANA timezone.
  7. Strategy states:
    - DRAFT
    - ACTIVE
    - PAUSED
    - CLOSED
  8. Monthly cycle states:
    - SCHEDULED
    - WAITING\_FOR\_MORTGAGE
    - WAITING\_FOR\_HELOC
    - HELOC\_DRAW\_PENDING
    - HELOC\_DRAW\_CONFIRMED
    - BROKERAGE\_TRANSFER\_PENDING
    - BROKERAGE\_FUNDED
    - ORDER\_PENDING
    - ORDER\_FILLED
    - RECONCILING
    - COMPLETED
    - PAUSED
    - FAILED
  9. Money-movement states:
    - REQUESTED
    - PENDING
    - SETTLED
    - FAILED
    - UNKNOWN
    - REVERSED
  10. Investment order states:
    - CREATED
    - SUBMITTED
    - PARTIALLY\_FILLED
    - FILLED
    - CANCELLED
    - REJECTED
    - UNKNOWN
  11. Add uniqueness constraints for:
    - tenant and user identities,
    - one cycle per strategy and payment period,
    - provider transaction identity,
    - idempotency key,
    - webhook event identity,
    - ledger business event identity.
  12. Model account ownership explicitly.
  13. Do not store full real account numbers. Use aliases and simulated provider IDs.
  14. Ledger rows are append-only at the application layer.
  15. Add optimistic concurrency/version columns where concurrent updates could occur.
  16. Add indexes for all common tenant-scoped lookups.
  17. Ensure a strategy cannot reference accounts belonging to another tenant or user through domain validation.
  18. Explain which cross-table invariants cannot be enforced directly by PostgreSQL foreign keys and where they are enforced.
- Create:
- Prisma schema
  - initial migration
  - typed repositories
  - transaction helper
  - tenant-scoped repository interfaces
  - seed data for two tenants
  - domain value objects
  - domain errors
  - tests proving tenant isolation and duplicate prevention

Do not implement Temporal workflows in this phase.

Run the migration against local PostgreSQL and execute all tests.

Implement the shared contracts package.

Create Zod schemas and inferred TypeScript types for:

- Tenant context
- Strategy setup
- Strategy activation
- Strategy pause and resume
- Mortgage account
- HELOC account
- Ordinary bank account
- Brokerage account
- Mortgage payment
- HELOC availability
- HELOC draw
- Bank transfer
- Brokerage deposit
- Investment order
- Investment fill
- Reconciliation result
- Monthly cycle status
- HELOC interest charge
- HELOC interest payment
- Provider webhook
- Simulator scenario configuration
- Stable API error envelope

Rules:

- Monetary values are integer cents.
- Validate positive and non-negative values correctly.
- Use discriminated unions for transaction and order states.
- Reject unknown properties at external boundaries.
- Define stable error codes.
- Include correlationId in API errors.
- Avoid exporting Prisma-generated models as public contracts.
- Separate internal domain types from HTTP contracts.
- Add contract serialization tests.
- Add examples used for OpenAPI documentation.

Create an account capability model:

Mortgage:

- canReadPayments

HELOC:

- canReadAvailability
- canDraw
- canReadInterestCharges

Ordinary bank account:

- canDebitInterest
- canReadTransactions

Brokerage:

- canDeposit
- canPlaceNotionalMarketOrder
- canReadOrder
- supportsFractionalUnits

Create tests for invalid money, invalid status transitions, malformed webhooks and unsupported account capabilities.

Implement the bank simulator as a realistic external provider.

The simulator must have its own provider-style data model and must not directly modify the application's core business tables.

Endpoints:

```
POST /sim/admin/scenarios
POST /sim/admin/users
POST /sim/admin/accounts
POST /sim/admin/run-events
POST /sim/admin/reset

GET /bank/accounts/:accountId
GET /bank/accounts/:accountId/transactions
GET /bank/mortgages/:mortgageId/payments
GET /bank/helocs/:helocId/availability
GET /bank/helocs/:helocId/interest-charges

POST /bank/helocs/:helocId/draws
GET /bank/helocs/:helocId/draws/:drawId
GET /bank/helocs/:helocId/draws/by-idempotency-key/:key

POST /bank/transfers
GET /bank/transfers/:transferId
GET /bank/transfers/by-idempotency-key/:key

GET /bank/ordinary-accounts/:accountId/debits/:debitId
```

The simulator must model:

- Mortgage payment lifecycle:  
SCHEDULED -> POSTED -> SETTLED  
and optional:  
POSTED -> REVERSED  
SETTLED -> REVERSED
- HELOC readvance:
- HELOC availability increases only after the configured delay following mortgage settlement.
  - Newly available credit must be represented by credit events.
  - Existing unused credit must be distinguishable from credit newly created during the payment period.
- HELOC draw:
- REQUESTED -> PENDING -> SETTLED  
or FAILED/UNKNOWN.
- HELOC interest:
- Monthly interest charge is created.
  - Payment is automatically debited from the configured ordinary bank account.
  - The ordinary account may have insufficient funds.
  - The interest payment is completely separate from the investment draw.

- Scenario configuration:
- mortgagePostingDelay
  - mortgageSettlementDelay
  - helocReadvanceDelay
  - drawSettlementDelay
  - transferSettlementDelay
  - interestChargeDay
  - interestDebitDelay
  - initialBalances
  - deterministic failure steps
  - seeded random failure rate
  - whether webhooks are enabled

- Failure scenarios:
- 429
  - transient 500
  - connection timeout before processing
  - timeout after successful processing
  - duplicate request
  - insufficient HELOC credit
  - ordinary bank account insufficient funds
  - delayed settlement
  - reversed mortgage payment
  - stale availability response
  - malformed webhook simulation

- Idempotency:
- Same key and same request returns the original operation.
  - Same key with different payload returns 409.
  - Persist idempotency results.
  - Support lookup by idempotency key.

- Webhooks:
- mortgage.payment.updated
  - heloc.availability.updated
  - heloc.draw.updated
  - bank.transfer.updated
  - heloc.interest-charged
  - heloc.interest-payment.updated

- Webhook delivery:
- signed payloads,
  - retries,
  - duplicate delivery,
  - out-of-order delivery,
  - optional dropped event.

- Add:
- OpenAPI documentation,
  - deterministic test clock abstraction,
  - scenario fixtures,
  - unit tests,
  - integration tests,
  - Docker health check.

The simulator must never import the application's Temporal workflows or repositories.



Implement the brokerage simulator as an independent provider-style service.

Endpoints:

POST /sim/admin/brokerage/accounts  
POST /sim/admin/brokerage/quotes  
POST /sim/admin/brokerage/scenarios  
POST /sim/admin/brokerage/run-events  
POST /sim/admin/brokerage/reset

GET /brokerage/accounts/:accountId  
GET /brokerage/accounts/:accountId/cash  
GET /brokerage/accounts/:accountId/positions

POST /brokerage/deposits  
GET /brokerage/deposits/:depositId  
GET /brokerage/deposits/by-idempotency-key/:key

POST /brokerage/orders  
GET /brokerage/orders/:orderId  
GET /brokerage/orders/by-idempotency-key/:key

The simulator supports:

- CAD non-registered accounts only for the MVP
- A configurable simulated ETF
- Notional market buy orders
- Fractional units
- Configurable quote price
- Configurable spread
- Optional commission
- Deposit settlement delay
- Order acknowledgement delay
- Fill delay
- Partial fill
- Rejected order
- Price movement between submission and fill
- Insufficient settled cash
- Account restriction
- Duplicate idempotency requests
- Timeout before processing
- Timeout after successful processing

Order lifecycle:  
CREATED  
SUBMITTED  
PARTIALLY\_FILLED  
FILLED  
REJECTED  
CANCELLED  
UNKNOWN

Deposit lifecycle:  
REQUESTED  
PENDING  
SETTLED  
FAILED  
UNKNOWN  
REVERSED

Require:

- Idempotency key for deposits and orders
- Same-key/same-request replay
- Same-key/different-request conflict
- Lookup by idempotency key
- Signed webhooks
- Duplicate and out-of-order webhook delivery
- Deterministic scenario mode
- Seeded random demo mode
- Safe decimal handling for quantity and price
- OpenAPI documentation
- Unit and integration tests

Webhook events:

- brokerage.deposit.updated
- brokerage.order.updated
- brokerage.fill.created
- brokerage.account.restricted

Do not connect Temporal directly to this simulator. Temporal Activities will use a brokerage client adapter later.



Implement robust typed clients for the simulated bank and brokerage APIs.

- Packages:
- packages/bank-client
  - packages/brokerage-client

- Requirements:
- Zod-validate every provider response.
  - Set correlation IDs.
  - Set idempotency keys for financial POST operations.
  - Configure explicit connection and response timeouts.
  - Classify errors as:
    - retryable transport failure,
    - rate limited,
    - provider unavailable,
    - validation failure,
    - business rejection,
    - duplicate conflict,
    - ambiguous result,
    - authentication failure.

Critical rule:  
A timeout after a financial POST is an ambiguous outcome. The client must not automatically repeat the POST.

Instead expose methods such as:

initiateHelocDraw(...)  
findHelocDrawByIdempotencyKey(...)  
resolveAmbiguousHelocDraw(...)

initiateTransfer(...)  
findTransferByIdempotencyKey(...)  
resolveAmbiguousTransfer(...)

submitOrder(...)  
findOrderByIdempotencyKey(...)  
resolveAmbiguousOrder(...)

Use bounded retry only for safe GET requests and explicitly safe operations.

- Add:
- fake adapters for unit tests,
  - provider contract tests,
  - error-mapping tests,
  - idempotency tests,
  - timeout-after-success tests,
  - redacted structured logs.

Do not put Temporal SDK dependencies inside these packages.



Implement the main application API.

For the simulator MVP, implement development authentication using signed JWTs with:

- tenantId
- userId
- roles

Roles:

- CUSTOMER
- OPERATIONS
- ADMIN

Do not build social login yet.

Endpoints:

POST /auth/dev-token

POST /strategies

GET /strategies

GET /strategies/:strategyId

PATCH /strategies/:strategyId

POST /strategies/:strategyId/activate

POST /strategies/:strategyId/pause

POST /strategies/:strategyId/resume

GET /strategies/:strategyId/cycles

GET /strategies/:strategyId/cycles/:cycleId

GET /strategies/:strategyId/ledger

GET /strategies/:strategyId/interest-payments

POST /financial-connections/simulated-bank

POST /financial-connections/simulated-brokerage

GET /financial-connections

GET /financial-accounts

POST /webhooks/bank

POST /webhooks/brokerage

Operations endpoints:

GET /operations/exceptions

GET /operations/cycles/:cycleId

POST /operations/strategies/:strategyId/resume

Requirements:

- Derive tenantId and userId from authentication.
- Never accept authorization scope solely from request body or URL.
- Enforce ownership and role checks.
- Verify webhook signatures using raw request body.
- Store webhook events before processing.
- Make webhook handling idempotent.
- Acknowledge valid webhooks quickly.
- Dispatch processing after persistence.
- Start, signal or query Temporal workflows through an application service.
- Reuse one Temporal client connection.
- Return stable error envelopes.
- Generate OpenAPI documentation.
- Add rate limiting.
- Add request-size limits.
- Add security headers.
- Add audit records for strategy activation, pause and resume.

Activation validation:

- mortgage exists and belongs to the user,
- HELOC exists and belongs to the user,
- ordinary bank account exists and belongs to the user,
- brokerage account exists and is non-registered,
- required capabilities are present,
- ETF policy is configured,
- monthly cap is valid,
- schedule timezone and payment date are valid,
- no account is already linked incompatibly.

Do not implement financial operations in HTTP handlers.

Implement Temporal Activities for the monthly conversion process.

Activity groups:

- Mortgage:
- findSettledMortgagePayment
  - identifyPrincipalRepaid
  - verifyPaymentNotReversed

- HELOC:
- getHelocAvailability
  - calculateNewlyAvailableCredit
  - initiateHelocDraw
  - resolveAmbiguousHelocDraw
  - confirmHelocDraw
  - getHelocInterestCharge
  - confirmInterestPayment

- Transfer:
- initiateBrokerageTransfer
  - resolveAmbiguousBrokerageTransfer
  - confirmBrokerageTransfer

- Brokerage:
- getSettledCash
  - submitInvestmentOrder
  - resolveAmbiguousInvestmentOrder
  - confirmInvestmentOrder
  - confirmInvestmentSettlement

- Cycle:
- loadStrategySnapshot
  - reserveMonthlyCycle
  - transitionCycleState
  - recordOperation
  - reconcileCycle
  - completeCycle
  - pauseStrategyWithException

- Audit:
- appendLedgerEntries
  - createAuditPackageMetadata

- Requirements:
- Activities are tenant-aware.
  - Activities reload authoritative state from the database.
  - Never trust critical account configuration supplied only by workflow input.
  - Validate strategy ownership and state on every financial Activity.
  - Financial Activities use provider idempotency keys.
  - Persist operation intent before provider POST when appropriate.
  - Persist provider references immediately after retrieval.
  - Use database transactions around state transitions.
  - Activities must be idempotent when executed repeatedly.
  - Use heartbeats for longer polling or reconciliation Activities where appropriate.
  - Define explicit Temporal retry policies by Activity category.
  - Mark business rejections non-retryable.
  - Do not automatically retry ambiguous financial results.
  - Resolve ambiguous results by provider lookup.
  - Redact sensitive information in errors.
  - Include correlationId, tenantId, strategyId and cycleId in logs.
  - Add unit tests for every Activity.
  - Add integration tests using simulator clients and PostgreSQL.

Do not implement the Workflow until Activity tests pass.

Implement MonthlyConversionWorkflow using the Temporal TypeScript SDK.

One Workflow execution represents one strategy payment period.

Workflow ID format:  
monthly-conversion/{tenantId}/{strategyId}/{paymentPeriod}

Input must contain only stable identifiers and non-sensitive scheduling data:

- tenantId
- strategyId
- paymentPeriod
- expectedPaymentDate
- timezone
- optional simulator scenario ID

Do not place provider credentials or full account details in Workflow input.

Workflow steps:

1. Load a strategy snapshot using an Activity.
2. Verify strategy is ACTIVE.
3. Reserve exactly one monthly cycle.
4. Move cycle to WAITING\_FOR\_MORTGAGE.
5. Check for the expected mortgage payment.
6. If not settled, wait for either:
  - a relevant bank-event Signal, or
  - a durable six-hour timer.
7. Repeat until settled or the 14-day business deadline expires.
8. Confirm the payment was not reversed.
9. Record actual principal repaid.
10. Move cycle to WAITING\_FOR\_HELOC.
11. Check actual HELOC availability and newly available credit.
12. Wait using Signal-or-six-hour-timer until enough new credit appears or the seven-day deadline expires.
13. Calculate the investment amount as the minimum of:
  - principal repaid,
  - newly available HELOC credit,
  - user cap,
  - platform cap.
14. If amount is below the configured minimum, complete as SKIPPED with a reason.
15. Initiate HELOC draw.
16. Resolve ambiguous results instead of blindly retrying.
17. Confirm draw settlement.
18. Transfer funds to brokerage.
19. Resolve ambiguous result if necessary.
20. Confirm brokerage deposit settlement.
21. Submit notional market buy order.
22. Resolve ambiguous result if necessary.
23. Confirm fill.
24. Confirm settlement.
25. Reconcile:
  - mortgage principal
  - > HELOC credit event
  - > HELOC draw
  - > brokerage transfer
  - > brokerage deposit
  - > investment fill.
26. Append ledger entries.
27. Mark cycle COMPLETED.

Signals:

- bankEventReceived
- brokerageEventReceived
- strategyPaused
- strategyClosed

Queries:

- getStatus
- getProgress
- getCurrentWaitReason

Workflow constraints:

- Deterministic code only.
- Use Temporal workflow time, sleep and condition APIs.
- Do not use Date.now directly unless through Temporal's deterministic API.
- Do not access database or network.
- Do not use Promise.race patterns that leave unmanaged waiting conditions.
- Reset event flags after consuming relevant Signals.
- Ignore duplicate or unrelated Signals safely.
- Never put entire provider webhook payloads into Workflow history.
- Pass only a small normalized event reference.
- Set bounded Activity timeouts.
- Use explicit retry policies.
- Financial POST Activities must have retry behaviour consistent with ambiguous-result resolution.
- Pause strategy on unreconciled or unsafe outcomes.
- Do not implement automatic financial rollback.
- Preserve provider references for operations review.

Create typed, stable machine-readable failure codes.

Add tests for:

- immediate settlement,
- three-day mortgage delay,
- HELOC readvance delay,
- webhook wakes polling early,
- duplicate Signal,
- unrelated Signal,
- payment reversal,
- insufficient credit,
- monthly cap,
- platform cap,
- amount below minimum,
- HELOC draw timeout after success,
- transfer timeout after success,
- order timeout after success,
- order rejection,
- partial fill,
- reconciliation mismatch,
- 14-day timeout,
- strategy paused during wait,
- strategy paused after draw,
- worker restart/replay determinism.

Use Temporal's time-skipping test environment where appropriate.

Implement Temporal Schedule management for active strategies.

Each active strategy receives a Schedule that starts one independent MonthlyConversionWorkflow per mortgage payment period.

Requirements:

- Use the strategy's IANA timezone.
- Support monthly mortgage payments for the MVP.
- Schedule the first check at 9:00 AM local time on the calendar day after the expected mortgage-payment day.
- Do not treat Schedule execution as proof that the mortgage payment occurred.
- The Workflow itself verifies settlement.
- Generate a stable paymentPeriod, such as 2026-08.
- Use a deterministic Workflow ID.
- Configure overlap policy to prevent overlapping cycles for the same strategy.
- Set catch-up behaviour explicitly.
- Handle month lengths when the configured day does not exist.
- Document the selected policy for dates such as February 30.
- Pause the Schedule when the strategy is paused.
- Unpause it when the strategy resumes.
- Delete or permanently pause it when the strategy closes.
- Schedule creation and updates must be idempotent.
- Keep a Schedule reference in the database.
- Reconcile database state against Temporal Schedule state.
- Add an administrative repair command.

Implement:

- createStrategySchedule
- updateStrategySchedule
- pauseStrategySchedule
- resumeStrategySchedule
- deleteStrategySchedule
- describeStrategySchedule
- reconcileStrategySchedules

Tests:

- activation creates one Schedule,
- repeated activation does not duplicate,
- timezone handling,
- month boundary,
- missing calendar day,
- pause/resume,
- duplicate trigger,
- overlap protection,
- catch-up after downtime,
- Schedule exists but database reference is missing,
- database reference exists but Schedule is missing.

Do not use OS cron or application setInterval.

Implement webhook processing that supplements polling but is never the only correctness mechanism.

- Bank webhook flow:
1. Verify signature.
  2. Parse and validate payload.
  3. Persist the provider event with a unique provider event ID.
  4. Return success quickly for valid events.
  5. Process the event idempotently.
  6. Find the relevant active strategy and payment period.
  7. Normalize the event to a small Signal payload:
    - providerEventId
    - accountId
    - eventType
    - providerResourceId
    - occurredAt
  8. Signal the existing MonthlyConversionWorkflow.
  9. If no workflow exists, retain the event for normal polling and reconciliation.
  10. Mark processing status.

Brokerage webhook flow follows the same pattern.

- Requirements:
- Duplicate webhooks must not duplicate financial actions.
  - Out-of-order events must be safe.
  - A Signal is only a wake-up hint.
  - The Workflow always calls an Activity to read current authoritative provider state.
  - Do not trust a webhook's status as final proof of settlement.
  - Do not send full provider payloads through Signals.
  - Handle SignalWithStart only if the architecture documentation explicitly justifies it.
  - Prefer signalling an already scheduled monthly workflow.
  - Add a dead-letter state for permanently invalid or unprocessable events.
  - Add retryable webhook-processing jobs without creating a separate external message broker.
  - Use a database outbox or durable processing table where needed.

- Tests:
- valid webhook,
  - invalid signature,
  - duplicate event,
  - out-of-order event,
  - webhook before workflow starts,
  - webhook after workflow completes,
  - unknown account,
  - cross-tenant account collision attempt,
  - dropped webhook with polling fallback.

Implement HelocInterestPaymentWorkflow.

This Workflow does not borrow or invest money.

- Purpose:
- Monitor each HELOC interest charge.
  - Verify that the associated payment comes from the user's configured ordinary bank account.
  - Record and reconcile the payment.
  - Pause or alert when the payment fails or uses an unexpected source.

- Start:
- Use a separate Temporal Schedule based on the expected monthly HELOC interest-charge period.
  - Start checking shortly after the expected interest charge date.
  - Do not assume the charge exists merely because the Schedule fired.

- Workflow:
1. Load strategy and account configuration.
  2. Check for the HELOC interest charge for the payment period.
  3. If absent, wait for Signal or six-hour timer.
  4. Once posted or settled, record the charge.
  5. Find the corresponding debit from the configured ordinary bank account.
  6. Confirm debit settlement.
  7. Verify:
    - amount,
    - HELOC destination,
    - ordinary account source,
    - payment period,
    - provider reference.
  8. Reconcile charge and payment.
  9. Append ledger/audit records.
  10. Complete.

- Failure conditions:
- insufficient ordinary-account funds,
  - debit rejected,
  - debit reversed,
  - charge missing after deadline,
  - amount mismatch,
  - unexpected payment source,
  - duplicate interest payment,
  - provider result unknown.

- Policy:
- Do not draw additional HELOC funds to cover interest.
  - Do not transfer brokerage money to pay interest.
  - Do not automatically move money from another account.
  - Create an operational exception and user alert.
  - Decide in the domain policy whether failed interest payment pauses only interest monitoring or also pauses future investment conversions.
  - Default: pause future conversion because the HELOC is not operating normally.

Add workflow, Activity, integration and time-skipping tests.



Implement financial reconciliation and the append-only ledger.

For every completed monthly cycle, prove the following chain:

1. Settled mortgage payment exists.
2. Principal amount is positive.
3. HELOC credit event is associated with the mortgage settlement period.
4. HELOC draw does not exceed the calculated investment amount.
5. Brokerage transfer matches the HELOC draw.
6. Brokerage deposit matches the transfer, accounting for explicitly supported fees only.
7. Investment order uses no more than settled brokerage cash.
8. Fill belongs to the expected order, account and symbol.
9. Remaining cash is explicitly recorded.
10. No provider transaction is already linked to another cycle.
11. No money movement crosses tenant or user boundaries.

Interest reconciliation chain:

1. HELOC interest charge.
2. Ordinary-bank-account debit.
3. Debit destination or reference identifies the HELOC.
4. Amount and period match.
5. Payment settled and was not reversed.

Ledger requirements:

- Append-only.
- Balanced debit and credit entries.
- Stable businessEventId.
- Account categories:
  - ASSET
  - LIABILITY
  - EQUITY
  - INCOME
  - EXPENSE
  - CLEARING
- Separate simulated provider records from accounting ledger records.
- Record currency.
- Store source provider references.
- Store cycleId and strategyId.
- Make repeated reconciliation idempotent.
- Never update or delete posted ledger entries.
- Correct mistakes using compensating ledger entries, not mutation.
- Add a ledger-integrity verification command.
- Add daily reconciliation reports.
- Add tests that total debits equal total credits.
- Add tests for duplicate provider transactions and mismatched amounts.

Produce a clear document explaining the accounting model using one \$770 example.



Implement the Next.js web interface.

Customer experience should be simple, but must clearly disclose that the strategy uses borrowed HELOC funds and creates investment debt.

Pages:

/login  
/onboarding  
/dashboard  
/strategy  
/activity  
/interest  
/documents  
/settings

Customer onboarding:

1. Select simulated bank scenario.
2. Connect simulated mortgage.
3. Connect simulated HELOC.
4. Connect ordinary bank account used for interest.
5. Connect simulated non-registered brokerage account.
6. Select one simulated ETF.
7. Configure monthly investment cap.
8. Review leverage and risk disclosure.
9. Activate automatic monthly conversion.

Dashboard:

- automation active/paused,
- next expected check,
- latest mortgage payment,
- principal repaid,
- latest amount borrowed,
- latest amount invested,
- investment-loan balance,
- HELOC interest paid from ordinary account,
- latest cycle status,
- exceptions requiring attention.

Use simple customer statuses:

- Waiting for mortgage payment
- Waiting for available credit
- Transferring funds
- Investing
- Confirming transactions
- Completed
- Paused

Do not expose raw Temporal terminology to customers.

Operations pages:

/operations/cycles  
/operations/exceptions  
/operations/webhooks  
/operations/reconciliation  
/operations/workflows

Operations view includes:

- detailed internal state,
- provider references,
- Activity attempts,
- error codes,
- reconciliation results,
- webhook history,
- safe retry or resume actions,
- Temporal Workflow link.

Requirements:

- Server-side authorization.
- Tenant isolation.
- No sensitive tokens in browser.
- Accessible forms.
- Responsive interface.
- Loading, empty and error states.
- Poll or use server-sent events for status updates.
- Do not add a separate message broker.
- Display amounts in CAD.
- Handle Canadian timezones.
- Add component and browser tests.

Act as a senior distributed-systems test engineer.

Inspect the complete system and implement a failure-focused test suite.

Test categories:

Settlement timing:

- payment settles immediately,
- payment takes one day,
- payment takes five days,
- payment never settles,
- payment reverses after posting,
- payment reverses after settlement.

HELOC:

- credit readvances immediately,
- credit readvances late,
- existing unused credit exists but no new credit event,
- credit insufficient,
- account blocked,
- account delinquent,
- draw rejected,
- draw timed out before processing,
- draw timed out after succeeding.

Transfer:

- transfer pending for several days,
- transfer rejected,
- transfer timed out after succeeding,
- duplicate request,
- reversed transfer.

Brokerage:

- delayed deposit,
- deposit reversed,
- insufficient settled cash,
- order rejected,
- partial fill,
- order timed out after fill,
- price moves,
- account restricted,
- symbol policy changed during cycle.

Temporal:

- Worker stops during each workflow stage,
- Temporal server restarts,
- Activity retries,
- duplicated Signals,
- out-of-order Signals,
- Schedule triggers twice,
- Workflow replay after code change test fixture,
- execution resumes without duplicate money movement.

Tenancy and security:

- user accesses another tenant's strategy,
- forged tenantId in request body,
- webhook references another tenant's account,
- reused provider ID across tenants,
- invalid webhook signature,
- replayed webhook,
- sensitive values absent from logs and Workflow input.

Interest:

- debit succeeds,
- insufficient ordinary-account funds,
- interest payment reversed,
- interest paid from unexpected account,
- duplicate debit.

Reconciliation:

- every amount mismatch,
- duplicate provider reference,
- missing fill,
- ledger imbalance,
- reconciliation rerun.

Use:

- unit tests,
- repository integration tests,
- provider contract tests,
- Temporal time-skipping tests,
- Docker Compose end-to-end tests.

Generate docs/test-matrix.md mapping every requirement and failure mode to automated tests.

Run the full suite and report:

- executed commands,
- passing tests,
- failing tests,
- coverage gaps,
- intentionally deferred cases.

Do not weaken assertions to make tests pass.

Implement production-style observability for the simulator.

- Requirements:
- Structured JSON logs.
  - OpenTelemetry traces.
  - Metrics for API, Activities, Workflows and provider simulators.
  - Correlation IDs propagated across:
    - HTTP request
      - > Temporal Client
      - > Workflow
      - > Activity
    - > bank/brokerage request
    - > database records.

- Metrics:
- active strategies,
  - cycles started,
  - cycles completed,
  - cycles paused,
  - settlement wait duration,
  - HELOC readvance wait duration,
  - transfer duration,
  - order-fill duration,
  - Activity retries,
  - ambiguous provider outcomes,
  - webhook duplicates,
  - reconciliation mismatches,
  - interest-payment failures,
  - Schedule reconciliation failures.

- Alerts for:
- financial operation in UNKNOWN state,
  - cycle stuck beyond threshold,
  - reconciliation mismatch,
  - repeated Activity failure,
  - interest debit failure,
  - Schedule missing,
  - ledger imbalance,
  - cross-tenant authorization attempt.

- Add:
- health endpoints,
  - readiness endpoints,
  - Temporal Worker identity and build information,
  - operations runbook,
  - incident-response steps,
  - safe restart procedure,
  - safe Workflow resume procedure.

- Do not log:
- provider tokens,
  - JWTs,
  - full account numbers,
  - raw webhook secrets,
  - personal tax information.

Create docs/operations-runbook.md.

Review all Temporal Workflow code for determinism and safe future evolution.

- Implement:
- replay tests using stored Workflow histories,
  - Workflow versioning strategy,
  - Worker deployment documentation,
  - safe changes policy,
  - Continue-As-New policy where applicable,
  - Workflow history-size monitoring,
  - build/version metadata,
  - compatibility tests.

The monthly conversion workflow is finite and may not normally need Continue-As-New. Explain why.

- Any long-lived entity or monitoring workflow must:
- check whether Continue-As-New is suggested,
  - pass compact state,
  - avoid carrying unnecessary event payloads,
  - ensure Signal/Update handlers have completed before continuing as new.

- Document changes that can break replay:
- adding or removing Activity calls,
  - changing timer ordering,
  - changing conditional command paths,
  - reordering awaited Temporal operations.

- Create:
- docs/temporal-versioning.md
  - replay test fixtures
  - CI replay-test command

Do not arbitrarily patch Workflow code. Explain every compatibility mechanism.

Create a complete deterministic demonstration scenario.

- User:
- Canadian simulated user
  - America/Edmonton timezone
  - Monthly mortgage payment on day 1

- Accounts:
- ordinary bank account with sufficient balance,
  - mortgage,
  - readvanceable HELOC,
  - non-registered brokerage account.

- Scenario:
- Mortgage payment: \$2,400.
  - Interest portion: \$1,630.
  - Principal portion: \$770.
  - Payment posts after 12 hours.
  - Payment settles after 48 hours.
  - HELOC credit readvances 12 hours after settlement.
  - Strategy monthly cap: \$1,000.
  - Platform cap: \$5,000.
  - Calculated investment amount: \$770.
  - HELOC draw settles after 2 hours.
  - Brokerage deposit settles after 4 hours.
  - Simulated ETF price: \$61.99.
  - Fractional market order fills.
  - Remaining cash is recorded.
  - HELOC interest is charged later and paid from the ordinary bank account.

- Create:
- seed command,
  - scenario command,
  - accelerated simulator clock,
  - end-to-end test,
  - README walkthrough,
  - screenshots or Playwright test evidence where possible.

- The demo must visibly prove:
- no investment occurs before mortgage settlement,
  - no investment occurs before HELOC readvance,
  - exactly one draw occurs,
  - exactly one transfer occurs,
  - exactly one order occurs,
  - retries do not duplicate operations,
  - the ledger balances,
  - the interest payment comes from the ordinary account,
  - customer dashboard reaches Completed.

Add a second scenario where the HELOC draw API times out after success. Prove that the system discovers the existing draw by idempotency key and does not submit another one.

Perform a final principal-engineer audit of the complete repository.

Do not modify code until completing the audit report.

Review:

Architecture:

- boundaries,
- coupling,
- Temporal usage,
- Schedule design,
- Workflow determinism,
- Activity idempotency,
- simulator independence.

Financial safety:

- integer money,
- decimal quantities,
- ambiguous result handling,
- duplicate prevention,
- account ownership,
- limits,
- pause conditions,
- reconciliation,
- ledger integrity.

Security:

- authentication,
- authorization,
- tenant isolation,
- webhook verification,
- secrets,
- logging redaction,
- input validation,
- SSRF and outbound URL configuration,
- dependency vulnerabilities.

Reliability:

- retries,
- timeouts,
- cancellation,
- worker restart,
- Schedule overlap,
- database transactions,
- concurrency,
- webhook duplication,
- out-of-order events.

Testing:

- requirement coverage,
- failure-path coverage,
- Temporal replay,
- integration realism,
- end-to-end tests.

Operations:

- metrics,
- traces,
- alerts,
- runbook,
- repair commands,
- safe deployments.

Produce:

- docs/final-audit.md
- Severity-ranked findings:
  - Critical
  - High
  - Medium
  - Low
- Evidence for every finding.
- Concrete remediation plan.
- Missing tests.
- MVP launch checklist.

Then fix all Critical and High findings.

Run the complete validation suite afterward.

Do not claim production readiness for real money. Explicitly describe the additional legal, security, provider-certification, privacy, compliance and operational work needed before real financial integrations.



Validate the work completed in the previous phase.

1. Inspect git diff.
2. Identify incomplete or placeholder implementation.
3. Check the implementation against the requested acceptance criteria.
4. Run:
  - formatting check,
  - lint,
  - TypeScript type checking,
  - unit tests,
  - relevant integration tests,
  - docker compose config,
  - relevant container health checks.
5. Check for:
  - TypeScript any,
  - unsafe type assertions,
  - floating-point money,
  - missing tenant filters,
  - secrets in source control,
  - unredacted sensitive logs,
  - non-deterministic Workflow code,
  - financial POST operations without idempotency,
  - blind retries after ambiguous outcomes,
  - tests that do not meaningfully assert behaviour.
6. Fix all issues within the current phase's scope.
7. Summarize:
  - files changed,
  - commands run,
  - results,
  - remaining risks,
  - intentionally deferred work.

Do not claim success unless the commands actually passed.